PRODUCT REQUIREMENTS & TARGET ARCHITECTURE

TimesPrime

A bilingual news aggregation platform — specified as built, and a structure to grow into

A reverse-engineered PRD derived from the code itself, paired with a feature-first codebase structure, a file-by-file migration map, and a phased implementation plan designed to change no behaviour.

PREPARED
29 October 2026

CODEBASE
Next.js 16.3 · React 19
~13,000 LOC · 105 files

BASELINE
commit `506a4ab` · branch main

METHOD
Source-derived, with file-level traceability

FRONT MATTER

How to read this document

This is a brownfield specification. It describes what TimesPrime *does*, not what someone once intended it to do — then proposes where the code should live.

That distinction drives every choice in here. A reverse-engineered PRD has a well-known failure mode: it reads the code, finds a behaviour, and writes it down as a requirement — including behaviours that are defects. Once a bug is printed in a requirements document it becomes extremely hard to remove, because every later reader assumes it was intended.

Three conventions guard against that:

- **Traceability.** Every functional requirement cites the file that implements it. A requirement with no citation is marked `PLANNED` — it is a wish, not a description.
- **Separation of defect from specification.** Behaviours that exist but should not are collected in §2.11 as *known defects*, explicitly excluded from the requirement set.
- **Compatibility requirements.** Brownfield work carries a constraint greenfield work does not: things that must keep working. These are numbered `CR-nnn` and are as binding as the functional requirements.

Document map

1. **Part I — Current state.** Product, stack, architecture, data flow, and an honest reading of the present structure with measurements.
2. **Part II — Product Requirements.** Personas, journeys, 24 functional requirements with traceability, non-functional requirements, compatibility constraints, data model, integrations, and the defect register.
3. **Part III — Target codebase structure.** The recommended structure and why, module responsibilities, the import rule that keeps it intact, and a complete file-by-file move map.
4. **Part IV — Implementation plan.** Six phases, the gate each must pass, slice ordering, and the specific risks that sink refactors like this one.
5. **Part V — Practices to adopt.** Conventions worth enforcing, and the ones this codebase already gets right.

> A NOTE ON WHAT IS ALREADY TRUE
>
> This codebase is better documented than most production systems. Non-obvious decisions carry comments that cite live data, record the alternatives considered, and explain why they were rejected. That is rare and genuinely valuable. Nothing in this document proposes discarding it — the structural work exists to make the rest of the codebase as legible as its comments already are.

PART I

Current state

1.1 What the product is

TimesPrime aggregates news from a commercial API, stores it on a rolling seven-day window, and serves a bilingual (English / Hindi) reading experience. It is not a thin API wrapper: between upstream and reader sit a filtering pipeline, a caching strategy, an article scraper, an AI classifier, an on-demand translator, and an editorial admin surface.

Two audiences use it. **Readers** browse a categorised feed, open an article to read scraped full text, switch language, save articles locally and ask an AI assistant about a story. **Operators** use `/admin` to moderate stored articles, publish editorial pieces, set breaking-news banners, trigger syncs and review an activity log.

1.2 Technology stack

LAYER	CHOICE	NOTES RELEVANT TO STRUCTURE
Framework	Next.js 16.3, App Router, Turbopack	Renames Middleware to **Proxy**; `priority` image prop deprecated for `preload`; `remotePatterns` hostnames glob-matched
UI	React 19, Tailwind CSS v4	Server Components by default; `"use client"` boundary is load-bearing
Language	TypeScript 5, `strict: true`	Compiles clean today
Database	Supabase (PostgreSQL) + RLS	Public read, service-role write; one table defined in committed SQL
Cache	Upstash Redis, in-process `Map` fallback	Fallback is per-instance — not shared on serverless
AI	Anthropic Claude Haiku	Two distinct uses: reader chat, and category classification
Scraping	axios + cheerio	~560 lines of extraction heuristics, no test coverage
Monitoring	Sentry, with a secret-scrubbing hook	Convention-located config files
Hosting	Vercel, cron in `vercel.json`	Cron path is a hard external contract

13 production dependencies, 15 development. That restraint is worth preserving.

1.3 Architecture and data flow

Three independent flows, each with its own failure posture.

```
INGEST — every 30 minutes, Vercel cron
  /api/cron/refresh-news  [Bearer CRON_SECRET, fails closed if unset]
    → NewsData.io, one request per category, sequential (bursts get throttled)
    → reject: hard-paywall domains · horoscope/tarot filler · wire-spam sources
              · non-news listing sources
    → upsert into Supabase on article_id
    → fire-and-forget: Claude Haiku classifies which category tags do not fit

SERVE — per reader request
  /api/news/top-headlines
    1. Redis cache          15 min TTL        → X-Data-Source: cache
    2. Supabase             if newest row < 30 min old → supabase
    3. NewsData.io          live call         → newsdata
    4. MOCK_ARTICLES        built-in fallback → mock
  Each tier degrades silently to the next. The header is the only signal
  of which tier actually answered.

AUTHORISE — every admin request
  src/proxy.ts   matcher: /admin/*, /api/admin/*
    → public: /admin/login, /api/admin/login, /api/admin/logout
    → public GET only: /api/admin/breaking-news  (homepage banner reads it)
    → otherwise: HMAC-SHA256 cookie, 12 h, httpOnly, SameSite=Lax
  No route handler performs its own auth check.
```

STRUCTURAL CONSEQUENCE

Authorisation is a single choke point. That is a good design — one place to audit, one place to change — but it means any route added outside the proxy matcher is silently public. The matcher has no test. A target structure must keep this property visible rather than scattering it.

1.4 The present structure, measured

DIRECTORY	LOC	FILES	WHAT ACTUALLY LIVES THERE
`src/app/api/`	2,324	17	Route handlers — several carry substantial business logic inline
`src/app/` (pages)	1,445	11	Pages and layouts
`src/components/`	5,483	17	Every component, flat, admin and reader mixed
`src/lib/`	2,909	20	**Six unrelated concerns in one folder**
`src/context/`	120	2	Theme and Language providers
`src/types/`	72	1	Shared types

The problem is `src/lib/`

It is not a module. It is a folder named after the absence of a decision, and it currently holds six genuinely different kinds of thing:

CONCERN	FILES
Data access	`db.ts` (571) · `supabase.ts`
Security primitives	`adminSession.ts` · `urlValidation.ts`
Cross-cutting infrastructure	`cache.ts` · `rateLimit.ts` · `sentryScrub.ts` · `adminLogger.ts`

Domain logic	`newsConfig.ts` · `categoryClassifier.ts` · `fetchNews.ts` · `newsdata.ts` · `paraphraser.ts` · `rssParser.ts`
Static configuration data	`imageDomains.ts` · `categoryColors.ts` · `topicImages.ts` (414)
Presentation / i18n	`translations.ts` · `googleTranslate.ts` · `bookmarks.ts`

Nothing in the folder name tells a reader which of those a given file is, and — more importantly — nothing prevents a React component from importing a Supabase client. That has already happened once: `ActivityLog.tsx` is a client component that imports `adminLogger.ts` for two constant objects and drags the entire Supabase SDK into the browser bundle with it.

Oversized units

Seven files exceed 500 lines, and none of the seven has any test coverage.

FILE	LOC	COVERAGE	DISTINCT RESPONSIBILITIES INSIDE
`components/ArticleDetailView.tsx`	1,008	0%	header · body · translation · AI panel · share · poll · related
`components/admin/ArticlesTable.tsx`	751	0%	columns · row actions · filters · pagination · bulk select
`lib/db.ts`	571	0%	filters · freshness · purge · upsert · stats
`components/Header.tsx`	568	0%	nav · search · theme · language · mobile menu · clock
`api/news/scrape-article/route.ts`	560	0%	fetch · paywall detect · extract · junk filter · persist
`components/HomeClient.tsx`	532	0%	feed state · polling · filters · grid · banner
`app/admin/page.tsx`	512	0%	tab shell · publish form · purge · seed

MEASURED, NOT ESTIMATED

Project-wide statement coverage is **4.41%**. Until recently the reported figure was 91.42% — Jest had no `collectCoverageFrom`, so it measured only the five modules the four test suites happened to import, or 6.8% of the source tree. The denominator is now honest. Every file in the table above reads zero.

PART II

Product Requirements

2.1 Overview

TimesPrime exists because raw news-API output is not a reading experience. The upstream feed carries duplicate headlines, horoscope filler tagged as business news, wire-spam accounts republishing the same crop-survey article under five categories, articles in scripts the product does not support, and paywalled stories whose "content" is a subscription prompt. The product's value is the editorial layer that removes all of that, plus the reading affordances layered on top: full text, translation, saving, and an assistant that can answer questions about a story.

It serves a bilingual Indian-market audience, with English and Hindi as first-class and equal — not a primary language plus a translation afterthought.

2.2 Goals and non-goals

GOALS	EXPLICIT NON-GOALS
A clean, de-duplicated, on-topic feed in two languages	Not a user-account product — no registration, no server-side user state
Readable article text without leaving the site	Not a CMS — editorial publishing is a thin escape hatch, not the main path
Serve readers from storage, not from the upstream API, to control cost	Not a social platform — no comments, sharing beyond the OS share sheet, or profiles
An operator surface for moderation and breaking news	Not multi-tenant — one publication, one admin credential
Degrade visibly rather than fail, on every third-party outage	Not real-time — a 30-minute refresh cadence is the design point

2.3 Personas

PERSONA	NEEDS	IMPLIED CONSTRAINTS
Reader	A fast, current, relevant feed in their language; full text without a paywall wall; to save things for later	No login. State is device-local. Must work on a mid-range phone on mobile data
Operator / editor	Remove bad articles; push a breaking banner within seconds; confirm the sync is alive; publish an original piece	Single shared credential. Every destructive action must be logged and reversible where possible

2.4 User journeys

1. **Browse the feed.** Land on `/` → server renders the category feed → client hydrates, polls for new stories every two minutes → a banner offers a refresh when newer rows exist.
2. **Read an article.** Tap a card → `/article/[slug]?url=&image=` → server calls the scraper → paywall detection, then `<p>` extraction, then JSON-LD fallback, then junk filtering → full text renders with a link to the publisher.

3. **Switch language.** Toggle to Hindi → the feed re-queries with a Devanagari script filter; on an article page the scraped text is translated on demand and cached per title.
4. **Save for later.** Bookmark → written to `localStorage` → a custom DOM event updates the header counter → `/saved` lists them.
5. **Ask the assistant.** Open the floating assistant, or ask from an article → `/api/chat` streams a Claude Haiku response, with the article title and description as context.
6. **Operate.** Sign in → proxy issues an HMAC session cookie → moderate, publish, set a banner, trigger a sync → each action appended to the activity log.

2.5 Functional requirements

24 requirements, each traced to implementing code. Status reflects verified behaviour at commit `506a4ab`, not intent.

ID	REQUIREMENT	IMPLEMENTED IN	STATUS	NOTE
FEED AND INGESTION				
FR-001	Fetch headlines by category from NewsData.io	`lib/newsdata.ts`, `lib/fetchNews.ts`	IMPLEMENTED	
FR-002	Refresh all 7 categories on a 30-minute schedule	`api/cron/refresh-news/route.ts`, `vercel.json`	IMPLEMENTED	Sequential by design — parallel bursts get throttled
FR-003	Serve from cache → store → upstream → mock, in that order	`api/news/top-headlines/route.ts`	PARTIAL	Tier 2 unreachable while the DB is down; see DEF-01
FR-004	Persist articles on a rolling 7-day window	`lib/db.ts` `purgeOldArticles`	PARTIAL	Purge runs on the read path; see DEF-05
FR-005	De-duplicate by normalised title	`lib/newsConfig.ts` `dedupeArticlesByTitle`	IMPLEMENTED	Covered by tests
FR-006	Exclude horoscope/tarot filler, wire-spam and non-news listing sources	`lib/newsConfig.ts`	IMPLEMENTED	Source lists evidence-backed in comments
FR-007	Exclude hard-paywalled publishers at write time	`lib/db.ts` `HARD_PAYWALL_SOURCE_DOMAINS`	IMPLEMENTED	Deliberately narrow; metered sites handled at read time
FR-008	Classify and suppress mis-tagged categories via LLM	`lib/categoryClassifier.ts`	IMPLEMENTED	Fail-open: unclassified is treated as "fits"
FR-009	Notify the reader when newer stories exist	`api/news/check-new/route.ts`, `HomeClient.tsx`	IMPLEMENTED	Polls on a 2-minute interval, pauses when tab hidden
LANGUAGE				
FR-010	Restrict each feed to its language by script detection	`lib/newsConfig.ts`, `lib/db.ts`	IMPLEMENTED	Non-Devanagari Indic scripts excluded from both feeds

FR-011	Translate article text on demand	`api/translate/route.ts`, `lib/googleTranslate.ts`	PARTIAL	Unofficial endpoint, no SLA; no input size cap
FR-012	Persist the language choice across visits	`context/LanguageContext.tsx`	IMPLEMENTED	
ARTICLE READING				
FR-013	Extract readable full text from a publisher page	`api/news/scrape-article/route.ts`	IMPLEMENTED	<p> extraction with a JSON-LD `articleBody` fallback
FR-014	Detect paywalls and show publisher metadata instead of junk	same — `detectPaywall`	IMPLEMENTED	DOM, meta-tag and schema.org signals
FR-015	Strip author bios, nav and app-download boilerplate	same — `isJunkParagraph`	IMPLEMENTED	Bilingual pattern set
FR-016	Refuse to fetch internal/private network addresses	`lib/urlValidation.ts`	PARTIAL	Guard is sound; redirects bypass it — DEF-02
FR-017	Always link out to the original publisher	`components/ArticleDetailView.tsx`	PARTIAL	Two of four links unvalidated — DEF-03
READER FEATURES				
FR-018	Save articles device-locally and list them	`lib/bookmarks.ts`, `app/saved/page.tsx`	IMPLEMENTED	Covered by tests
FR-019	Answer questions about a story, streamed	`api/chat/route.ts`	IMPLEMENTED	Covered by tests; error path misleads — DEF-04
FR-020	Provide a themed, topic-appropriate image for every card	`lib/topicImages.ts`, `lib/imageDomains.ts`	IMPLEMENTED	Per-topic round robin avoids repeats
OPERATIONS				
FR-021	Gate every admin surface behind a session	`src/proxy.ts`, `lib/adminSession.ts`	PARTIAL	Sound crypto, but unthrottled — DEF-06
FR-022	List, search, edit, delete and bulk-delete stored articles	`api/admin/articles/*`, `admin/ArticlesTable.tsx`	IMPLEMENTED	
FR-023	Publish an editorial article directly	`api/admin/publish-article/route.ts`	IMPLEMENTED	Bypasses the filter pipeline by design
FR-024	Set a time-boxed breaking-news banner	`api/admin/breaking-news/route.ts`	BLOCKED	Table absent from committed schema — DEF-07

2.6 Non-functional requirements

ID	REQUIREMENT	CURRENT REALITY

NFR-01	A reader request should not call the upstream API	**Not met.** Every uncached request reaches NewsData.io because tier 2 is unavailable
NFR-02	No third-party outage produces a user-visible error	Met — arguably over-met; see §2.11 on silent degradation
NFR-03	Abuse-prone endpoints are rate-limited	Partial — chat and translate only; login and scraper unprotected
NFR-04	Request bodies and upstream responses are size-capped	**Not met.** No cap on chat message, translate array, or scraped response
NFR-05	Secrets never reach the browser or an error report	Met — `sentryScrub.ts` redacts; no secret is `NEXT_PUBLIC_`
NFR-06	Stored articles are purged after 7 days	Met, but on the wrong execution path
NFR-07	The app is usable at 400 px width	Appears met from the breakpoint set; not verified in a browser
NFR-08	Typecheck, lint, test and build all pass on every commit	Met as of this baseline, and now enforced in CI
NFR-09	Dependencies carry no known critical advisory	**Not met.** 2 critical, 9 high — including the framework itself
NFR-10	Business logic is covered by tests	**Not met.** 4.41% statements

2.7 Compatibility requirements

Brownfield constraints. These bind the restructure as firmly as any functional requirement — each one, if broken, breaks something outside the codebase.

ID	MUST NOT CHANGE	WHY
CR-01	Every public URL, byte for byte	Existing links, search indexing, any external embed
CR-02	`/api/cron/refresh-news`	Hard-coded in `vercel.json`; ingestion stops silently if moved
CR-03	The `articles` table shape	Live data exists; no migration tooling is in place
CR-04	Every `process.env` name	Set in the Vercel dashboard, outside version control
CR-05	`src/proxy.ts` location and matcher semantics	Framework-pinned, and the sole authorisation boundary
CR-06	`imageDomains.ts` importable from `next.config.ts`	Evaluated at config-load time — it must never import React or `server-only`
CR-07	Sentry and instrumentation file locations	Framework conventions; silently inert if moved
CR-08	Public GET on `/api/admin/breaking-news`	The homepage banner has no session and reads it directly

2.8 Data model

One table is defined in committed SQL. Two more are used by code and exist only as comments inside route files — the single most consequential gap in the repository, because a fresh deployment appears to succeed and then silently lacks two features.

TABLE	DEFINED IN	STATUS	SHAPE

`articles`	`db/schema.sql`	COMMITTED	15 columns. `article_id` unique. `category` / `country` / `category_mismatches` are `TEXT[]`. Only `id`, `article_id`, `title`, `created_at` are NOT NULL
`breaking_news`	route-file comment	NOT COMMITTED	`id`, `text`, `is_active`, `expires_at`, `created_at`
`admin_logs`	route-file comment	NOT COMMITTED	`id`, `action`, `details`, `metadata` (JSONB), `created_at`

Indexes. Two exist (`created_at`, `article_id`). Queries also filter on `category` and `country` (array containment — wants GIN) and on `title` / `description` with `ILIKE '%…%'` (cannot use a B-tree at all — wants `pg_trgm`). Not urgent at a seven-day volume; a cliff as retention grows.

2.9 Integrations and failure behaviour

SERVICE	PURPOSE	ON FAILURE	ASSESSMENT
NewsData.io	Source of all articles	Falls back to 5 built-in mock articles	Correct for a feed; the mock set is obviously synthetic
Supabase	Storage, 7-day window	Silently serves upstream instead	**Too quiet.** The failure is invisible and expensive
Upstash Redis	Shared cache	In-process `Map`	Correct, and the trade-off is documented
Anthropic	Chat + classification	Chat returns 503; classifier fails open	Fail-open on classification is the right call
Google Translate	On-demand translation	Returns the original string	Unofficial endpoint — acceptable at this volume, not a foundation
Sentry	Error reporting	No-op without a DSN	Fine

2.10 Success metrics

METRIC	TARGET	WHY THIS NUMBER
Reader requests served without an upstream call	≥ 95%	The entire storage tier exists for this; currently ~0%
Upstream calls per day	≤ 350	7 categories × 48 cron runs, plus headroom
Feed items surviving the filter pipeline	30–40 per category	Below 30 suggests over-filtering; see DEF-08
Scrape success (real text, not fallback copy)	≥ 85%	Measured at ~96% in a 120-URL sample
Statement coverage on non-UI modules	≥ 60%	From 4.41%; security and data modules first
Critical/high dependency advisories	0	Currently 2 critical, 9 high

2.11 Known defects — explicitly not requirements

> **WHY THIS SECTION EXISTS**
>
> Everything below is current, observable behaviour. A naive reverse-engineered PRD would have written each one up as a requirement. They are defects, listed here so no future reader mistakes them for intent.

ID	BEHAVIOUR	SEVERITY	WHY IT IS A DEFECT, NOT A FEATURE
DEF-01	Every Supabase call fails; app silently serves upstream	CRITICAL	Defeats the caching architecture and burns quota, with no signal anywhere
DEF-02	Scraper follows up to 21 redirects, none re-validated	CRITICAL	Defeats the SSRF guard that runs immediately before it
DEF-03	Two of four publisher links render an unvalidated URL	HIGH	Branded phishing surface. The other two call sites already guard correctly
DEF-04	AI panel prints a canned "Key Takeaway" when the API fails	HIGH	Presents a fabricated line as a real answer. The chat widget handles the same failure honestly
DEF-05	A `DELETE` runs on the feed read path	MEDIUM	Retention is a scheduled job's work, not a reader's
DEF-06	Admin login has no rate limit; password compare leaks length	HIGH	Unlimited guessing against a single shared credential
DEF-07	Two tables exist only in code comments	HIGH	A fresh deploy silently lacks breaking news and the activity log
DEF-08	Feed fetches 80 rows, filters, then slices to 40	MEDIUM	Filtering after limiting under-fills precisely when a spam source floods
DEF-09	Contact form and newsletter show success, send nothing	MEDIUM	No endpoint exists. The user is told a message was delivered
DEF-10	Scraper returns HTTP 200 with placeholder prose on failure	MEDIUM	Forces the UI to detect errors by string-matching three hard-coded sentences

2.12 Open questions for the product owner

1. Is the **paraphraser** — which rewrites every headline through a synonym table — a deliberate republishing posture, or a legacy experiment? It changes the meaning of every stored title and has no owner in the code.
2. Should **RSS** return? 233 lines sit commented out pending publisher licensing, with a live endpoint that only returns 503. Either revive or delete.
3. Is **editorial publishing** a real workflow or an admin convenience? It currently bypasses every filter and tags everything `country: ["us"]`.
4. What is the intended **contact** path? The form implies one exists.
5. Should the feed admit languages beyond English and Hindi? Odia, Bengali and Tamil articles arrive today and are discarded.

PART III

Target codebase structure

3.1 What the framework actually prescribes

> FROM THE BUNDLED NEXT.JS 16.3 DOCUMENTATION
>
> "Next.js is **unopinionated** about how you organize and colocate your project files." The documentation presents three strategies — project files outside `app`, inside `app`, or split by feature — and concludes: "choose a strategy that works for you and your team and be consistent across the project."

So the structure is a team decision, not a framework mandate. What the framework *does* fix: files named `page`, `layout`, `route`, `loading` or `error` under `src/app/**` **are** the routing table; route groups `(name)` reorganise without touching URLs; private folders `_name` opt out of routing entirely; and a route is not public until a `page` or `route` file exists, which is what makes colocation safe.

3.2 Why feature-first, for this project specifically

Industry practice has converged on feature-first grouping for App Router projects, with Feature-Sliced Design as a heavier variant. But the general argument matters less than the specific one, and this codebase makes it on its own:

- **The pain is already concrete.** `src/lib/` holds six unrelated concerns and nothing distinguishes them. A client component has already imported a Supabase client through it.
- **The features are genuinely separable.** News, article, admin, chat, bookmarks and i18n barely overlap. This is not a domain where everything touches everything.
- **The riskiest code is the least isolated.** The scraper's 560 lines of heuristics sit inside a route handler, where they cannot be tested without constructing an HTTP request.
- **Deletion is the real test.** Removing the AI assistant today means touching a route, two components and a layout. It should mean deleting one folder.

The proposal below is deliberately *lighter* than full FSD. FSD's layer taxonomy (shared/entities/features/widgets/pages) carries ceremony that a 13,000-line codebase with two personas does not need, and over-structuring is as costly as under-structuring.

3.3 Proposed structure

```
timesprime/
├── db/                     schema.sql · policies.sql          ← already moved
├── docs/                   PRD · architecture · adr/ · runbooks/ · BACKLOG
├── scripts/                operational one-offs
├── public/
└── src/
    ├── app/                ROUTING ONLY — no business logic below this line
    │   ├── (reader)/       / · article/[slug] · saved        ← route group, URLs unchanged
    │   ├── (marketing)/    about · contact · privacy · terms
    │   ├── admin/          login · dashboard
    │   └── api/            paths frozen — see CR-01, CR-02
    │
    ├── features/           ONE FOLDER PER CAPABILITY
    │   ├── news/           feed fetching · source config · filters · classification
    │   ├── article/        detail view · scraping · extraction · paraphrasing
    │   ├── admin/          dashboard · tables · logs · breaking news
    │   ├── chat/           AI assistant
    │   ├── bookmarks/      device-local saved articles
    │   └── i18n/           language context · translations · translate client
    │       └── each: components/ · server/ · types.ts · index.ts  ← the ONLY public surface
    │
    ├── components/ui/      presentational primitives, zero domain knowledge
    ├── server/             db client · supabase · adminSession · adminLogger
    ├── lib/                cross-cutting only: cache · rateLimit · urlValidation · sentryScrub
    ├── config/             constants · imageDomains · categoryColors · topicImages
    └── types/              genuinely shared types only

PINNED BY THE FRAMEWORK — these do not move:
    src/proxy.ts · src/instrumentation.ts · src/instrumentation-client.ts
    src/sentry.server.config.ts · src/sentry.edge.config.ts
```

3.4 Module responsibilities

MODULE	OWNS	MUST NEVER
`app/`	URL structure, request/response shape, auth delegation. Route handlers stay thin — parse, call a feature, serialise	Contain extraction heuristics, filter logic or SQL
`features/news/`	Upstream fetching, source allowlists, script and spam filters, category classification, freshness	Import from another feature
`features/article/`	Scraping, paywall detection, extraction, junk filtering, the detail view	Know about admin or chat
`features/admin/`	Operator UI and its server actions	Be imported by any reader-facing code
`features/chat/`	Assistant UI, prompt assembly, streaming	Reach into article internals — take context as a parameter
`features/bookmarks/`	Device-local persistence and its UI	Touch the server at all
`features/i18n/`	Language state, static strings, translation client	Hold feature-specific copy
`components/ui/`	Buttons, cards, skeletons, spinners — appearance only	Import a feature, or know what an Article is
`server/`	Database client, session issuing/verification, audit logging	Be imported by a client component
`lib/`	Genuinely cross-cutting, domain-free: cache, rate limit, URL validation, scrubbing	Become the next dumping ground — if it has a domain, it belongs in a feature

`config/`	Static data: allowlists, colour maps, topic images, constants	Import React, `server-only`, or anything else (see CR-06)

3.5 The one rule that keeps this intact

> **THE BOUNDARY RULE**
> Each feature exposes exactly one public surface: `features/<name>/index.ts`. Nothing imports across feature boundaries. Features import only **downward** — into `components/ui`, `lib`, `server`, `config`. **If two features need the same thing, it moves down. It does not get imported sideways.**

Sideways imports are precisely how a feature-first structure collapses back into the thing it replaced, usually within a quarter and always one reasonable-looking import at a time. Documentation does not prevent this; a linter does. Phase 5 makes the rule machine-enforced via `no-restricted-imports` or `eslint-plugin-boundaries`, so a violation fails CI rather than passing review.

3.6 Move map

Every current module, with its destination and the judgement behind it.

CURRENT	DESTINATION	REASONING
`lib/db.ts`	`server/db/`	Data access. Split by concern during Phase 6
`lib/supabase.ts`	`server/supabase.ts`	Server-only client
`lib/adminSession.ts`	`server/auth/`	Security primitive; Node `crypto`
`lib/adminLogger.ts`	`server/auditLog.ts`	Server-only. Its two constant maps move to `features/admin` — this is the fix for the client-bundle leak
`lib/urlValidation.ts`	`lib/urlValidation.ts` *(stays)*	Genuinely cross-cutting, domain-free
`lib/cache.ts`, `rateLimit.ts`, `sentryScrub.ts`	*stay in* `lib/`	Infrastructure with no domain
`lib/newsConfig.ts`	`features/news/`	**Judgement call.** Looks like config, but encodes editorial policy — which sources are spam, which scripts belong. Policy is domain logic. It also carries the evidence comments, which belong with the feature
`lib/categoryClassifier.ts`	`features/news/server/`	Domain logic, unambiguous
`lib/fetchNews.ts`, `newsdata.ts`	`features/news/server/`	Upstream client
`lib/paraphraser.ts`	`features/article/`	Transforms article text. Pending the §2.12 question on whether it survives
`lib/rssParser.ts`	*delete*	233 commented-out lines. Git history preserves it
`lib/topicImages.ts`	`config/topicImages.ts`	**Judgement call.** 414 lines, but almost all of it is a keyword→URL table. The selection function is thin. Data, not logic

`lib/imageDomains.ts`	`config/imageDomains.ts`	**Handle first and carefully** — `next.config.ts` imports it at config-load time (CR-06)
`lib/categoryColors.ts`	`config/categoryColors.ts`	Static map
`lib/translations.ts`, `googleTranslate.ts`	`features/i18n/`	i18n is a feature, not a utility
`lib/bookmarks.ts`	`features/bookmarks/`	Self-contained, already tested — ideal first slice
`context/LanguageContext.tsx`	`features/i18n/`	Belongs with what it serves
`context/ThemeContext.tsx`	`components/ui/theme/`	Presentational, used everywhere
`components/ArticleDetailView.tsx`	`features/article/components/`	Then decomposed in Phase 6
`components/HomeClient.tsx`, `ArticleCard`, `MarqueeTicker`, `CategoryMegaMenu`	`features/news/components/`	Feed surface
`components/admin/*`	`features/admin/components/`	Whole folder moves intact
`components/Chatbot.tsx`, `FloatingAiAssistant.tsx`	`features/chat/components/`	
`components/Header.tsx`, `Footer.tsx`	`components/ui/layout/`	Used by every surface. Header's search and language controls move to their features in Phase 6
`components/ThemeSpinner`, `NewsSkeletonGrid`	`components/ui/`	Pure presentation
`components/BreakingNewsBanner.tsx`	`features/admin/components/`	**Judgement call.** Reader-facing, but its entire contract is the admin breaking-news API (CR-08). Owned by the feature that produces the data
`api/news/scrape-article/route.ts`	logic → `features/article/server/`; route stays a thin handler	The highest-value extraction in the project: 560 untestable lines become testable ones
`src/proxy.ts`, `instrumentation*.ts`, `sentry.*.config.ts`	*stay — framework-pinned*	CR-05, CR-07

PART IV

Implementation plan

Six phases. Each ends at a gate; nothing proceeds past a red one. The sequencing is deliberate: correctness before structure, structure before decomposition.

PHASE	WORK	EXIT GATE	WHY HERE IN THE ORDER
0	Baseline — record all four verifications, capture the route table	All four green DONE	A refactor cannot be verified against an unknown starting point
1	Hygiene — package name, scripts, README, CI, `db/`, docs scaffold	CI green DONE	Zero app code. Establishes the gate everything later depends on
2	PRD — this document's Part II, reviewed and approved	Owner sign-off, §2.12 answered	Structure follows from what the product is. Reversing this means guessing
3	Structure design — tree, move map, ADR recording rejected alternatives	Tree and map approved	Disagreement is cheap on paper and expensive after 300 files have moved
4	Migration — ten slices, pure moves only	Four green per slice; route table byte-identical	The bulk of the work, and the least risky if sequenced correctly
5	Enforcement — boundary lint rule, path aliases, documented import rules	A deliberate violation fails CI	Without this, phase 4 decays within a quarter
6	Decomposition — split the seven oversized files	Characterization test before each split	Last, because splitting untested code is how refactors cause outages

4.1 Migration slice order

Least coupled first, so the early slices build confidence in the mechanics before the hard ones.

#	SLICE	RISK	WATCH FOR
1	`config/` extraction	HIGH	**Despite being first.** `imageDomains.ts` is imported by `next.config.ts` at config-load time. Move the import in the same commit; confirm `remotePatterns` still resolves (CR-06)
2	`server/` extraction	MEDIUM	Verify nothing client-side imports these afterwards
3	`features/bookmarks`	LOW	Self-contained and tested. The slice that proves the mechanics
4	`features/i18n`	LOW	Context provider must stay above its consumers in the tree
5	`features/chat`	LOW	Has tests; the route must keep streaming
6	`features/news`	MEDIUM	Largest surface; touches the most routes
7	`features/article`	HIGH	Extracting scraper logic is not a pure move — see the stop condition below

8	`features/admin`	MEDIUM	Must not disturb the proxy matcher (CR-05)
9	`components/ui`	LOW	Whatever genuinely remains shared
10	`app/` route groups	HIGH	URLs must be byte-identical. Diff against the Phase 0 capture

4.2 Rules of engagement

- **A commit is either a move or a change, never both.** Use `git mv` so rename detection works. Mixing them breaks `git blame` and `log --follow` across the commit, which is exactly when you most need them.
- **Every slice runs all four verifications** before it is committed. Each catches a different class: `tsc` finds imports that no longer resolve, `eslint` finds the ones left behind, `jest` covers the thin tested slice, `build` catches route resolution and server/client boundary violations the others miss.
- **A red slice is reverted, not patched.** Stacking a fix on a broken slice hides what broke.
- **Stop condition.** If a slice cannot be completed as a pure move and needs an edit to a module body, stop. Either write a characterization test first, or defer that file to Phase 6 and say so explicitly. Slice 7 will probably hit this.
- **Defer, don't fix.** Anything tempting found mid-move goes to `docs/BACKLOG.md` with its reason, not into the commit.

4.3 Work that is not restructuring

These are independent of the phases above and should not wait for them. Ordered by consequence ÷ effort.

ORDER	ACTION	EFFORT	WHY NOW
1	Upgrade Next.js to 16.4.3	Small	Patch bump; clears four critical unauthenticated-RCE advisories
2	Restore Supabase; add `/api/health`	Small	DEF-01 is both the top correctness and top cost issue
3	Promote the two tables into `db/schema.sql`	Small	DEF-07; silent feature loss on any fresh deploy
4	Guard the two unvalidated publisher links	Small	DEF-03; two call sites already show the correct pattern
5	Rate-limit login and scraper; cap request bodies	Small	DEF-06, NFR-03, NFR-04
6	Remove the fabricated AI fallback and success-on-failure messages	Small	DEF-04, DEF-09; the product should not state falsehoods
7	Revalidate redirects in the scraper	Medium	DEF-02; needs manual hop handling
8	Tests for `urlValidation` and `adminSession`	Medium	~190 lines, pure, security-critical, currently untested

PART V

Practices to adopt

5.1 Keep doing

- **Comments that carry evidence.** This codebase records why a decision was made, what the data showed, and which alternatives were rejected. That is the single most valuable thing in it. Preserve these comments verbatim through every move.
- **Deliberate failure posture.** Fail-open on classification, fail-closed on the cron secret — each documented with its reasoning. Keep making that choice explicitly.
- **Narrow, evidence-backed blocklists.** Sources were excluded only after checking they had no legitimate content. Several candidates were rejected for false-positive risk. Hold that bar.
- **A small dependency tree.** 13 production dependencies. Every addition is a permanent liability.

5.2 Start doing

PRACTICE	WHAT IT MEANS HERE
Fail loudly on writes	Fail-open suits a news feed with a truthful fallback. It is wrong for writes, forms, AI answers and admin actions. The rule: **degrade silently only on read paths that have an honest fallback.**
Health checks over inference	A `/api/health` reporting each dependency would have surfaced DEF-01 immediately instead of hiding it behind a working-looking page
Status codes, not prose	A failed scrape returns a status, not a sentence the UI must recognise by string match
Machine-enforced boundaries	A documented import rule is a suggestion. A lint rule is a boundary
Characterization before extraction	Capture current behaviour as a test, then refactor until it still passes. Mandatory for all seven oversized files
Thin route handlers	Parse, delegate, serialise. Logic lives in a feature where it can be tested without an HTTP request
Schema in version control	Every table the code touches is defined in `db/`. No exceptions, no comments-as-schema
Read the bundled docs	Next 16 differs from most published material. `node_modules/next/dist/docs/` is authoritative; memory is not

5.3 Definition of done

For any change to this codebase:

1. `npm run verify` passes — typecheck, lint, test, build.
2. The commit is a move *or* a change, not both.
3. New logic outside a React component carries a test.
4. Anything deferred is written down in `docs/BACKLOG.md`, with its reason.
5. Any non-obvious decision carries a comment explaining why, in the style the codebase already uses.

5.4 Closing assessment

TimesPrime is a well-designed application with an operational-hardening deficit. The ingestion pipeline is thoughtful, the filtering is evidence-driven, and the failure handling is deliberate rather than accidental. Those are the hard parts, and they are done.

What is missing is the scaffolding that keeps a system healthy as it grows: a structure that makes the right import easy and the wrong one impossible, tests on the modules where a mistake is expensive, and signals loud enough that a dead database cannot masquerade as a working feed for days.

None of that requires rewriting what exists. It requires moving it somewhere it can be defended, and that is what the plan in Part IV does.

Sources consulted. Bundled Next.js 16.3 documentation (`node_modules/next/dist/docs/01-app/01-getting-started/02-project-structure.md`) for framework conventions and the three organisation strategies; the project's own source, schema, git history and toolchain output for every measurement; industry practice on feature-first App Router structure and on brownfield PRD method — specifically FR/NFR/CR numbering, code-evidence traceability, and the warning that reverse-engineered specifications capture defects as requirements unless explicitly separated. All figures in this document were measured against commit `506a4ab` , not estimated.

Limits. Responsive layout and accessibility were reviewed at code level only, not in a browser. Database behaviour could not be exercised because the Supabase connection is failing (DEF-01). Production bundle size was not measured. Per-advisory exploitability of the Next.js CVEs was not individually confirmed — the patch upgrade makes the question moot.